TP 04 — Fourmis et phéromones

Inside ESILV : une journée dans la peau d'un élève-ingénieur

Sixtine Michel

De Vinci Higher Education — ESILV

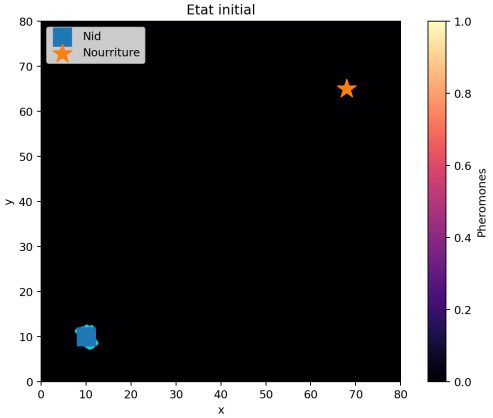
ESILV
DE VINCI ENGINEERING SCHOOL

Notre mission

Objectif

Faire émerger une piste entre un nid et une source de nourriture, sans donner sa position aux fourmis.

Chaque fourmi applique seulement des règles locales : explorer, sentir, transporter et déposer.



D'un point de vue algorithmique

À chaque image, les fourmis et leur environnement évoluent ensemble :

```
def creer_fourmis():
    # Initialiser les fourmis autour du nid.
def creer_champ_pheromones():
    # Creer une grille sans pheromone.
def deplacer_fourmis(fourmis, champ):
    # Explorer, rapporter et deposer une piste.
def diffuser_et_evaporer(champ):
    # Etaler puis attenuer les pheromones.
```

La répétition de ces règles locales fait apparaître une piste collective.

Les données du programme

- `positions` et `directions` : tableaux de forme $(N, 2)$;
- `transporte` : un booléen par fourmi ;
- `reserve_pheromone` : quantité encore disponible ;
- `champ[x,y]` : concentration sur une grille 80×80 ;
- `pas_memoire_retour` : durée du cap appris.

Deux états

Cyan : recherche de nourriture. Violet : retour chargé vers le nid.

Vecteur et normalisation

Pour aller d'une position P vers une cible C :

$$\vec{v} = C - P, \qquad \|\vec{v}\| = \sqrt{v_x^2 + v_y^2}, \qquad \vec{u} = \frac{\vec{v}}{\|\vec{v}\|}.$$

Le vecteur unitaire $\vec{u}$ donne une direction sans modifier la vitesse.

Cas particulier

Si $\|\vec{v}\| = 0$, on renvoie le vecteur nul pour éviter une division par zéro.

Diffusion et évaporation

Chaque cellule reçoit une partie de la moyenne de ses quatre voisines :

$$P' = (1 - d)P + d\,\overline{P}_{\text{voisins}}, \qquad P_{\text{suivant}} = eP'.$$

Diffusion d

Étale légèrement la piste dans l'espace.

Évaporation $e < 1$

Efface progressivement les anciennes pistes.

Les cinq étapes de la simulation

1. créer les fourmis et faire évoluer le champ ;
2. calculer les directions unitaires et le gradient odorant ;
3. choisir une direction, avancer et rebondir ;
4. prendre ou livrer la nourriture ;
5. déposer une piste décroissante pendant le retour.

Correspondance entre étapes et TODO

1. Initialisation, diffusion et évaporation	**TODO 1 à 3**
2. Vecteurs unitaires et gradient	**TODO 4 à 7**
3. Direction, déplacement et rebonds	**TODO 8 à 12**
4. Prise et livraison de nourriture	**TODO 13 et 14**
5. Dépôt décroissant de phéromones	**TODO 15**

Directions initiales — TODO 1

À vous

- Creer NB_FOURMIS directions initiales aleatoires.

Directions initiales — TODO 1

```
angles = np.random.uniform(0, 2*np.pi, NB_FOURMIS)
directions = np.column_stack((np.cos(angles),
                              np.sin(angles)))
```

Chaque angle produit un point du cercle unité : $(\cos\theta, \sin\theta)$.

Faire évoluer le champ — TODO 2 et 3

À vous

- Calculer la moyenne des quatre cases voisines.
- Diffuser puis evaporer les pheromones.

Faire évoluer le champ — TODO 2 et 3

```
moyenne_voisins = (
    np.roll(champ, 1, axis=0) + np.roll(champ, -1, axis=0)
    + np.roll(champ, 1, axis=1) + np.roll(champ, -1, axis=1)
) / 4.0
champ[:] = ((1-DIFFUSION)*champ
            + DIFFUSION*moyenne_voisins)
champ *= EVAPORATION
```

Une direction unitaire — TODO 4 et 5

À vous

- Construire cible - position.
- Renvoyer le vecteur unitaire vecteur / norme.

Une direction unitaire — TODO 4 et 5

```
vecteur = cible - position
norme = np.linalg.norm(vecteur)
if norme == 0:
    return np.zeros(2)
return vecteur / norme
```

Sentir une piste — TODO 6 et 7

La fourmi compare les concentrations moyennes à droite/gauche et en haut/bas :

$$g_x = \overline{P}_{droite} - \overline{P}_{gauche}, \qquad g_y = \overline{P}_{haut} - \overline{P}_{bas}.$$

Puis elle normalise (g_x, g_y). La moyenne sur plusieurs cellules rend la détection robuste lorsqu'une piste est diagonale.

Sentir une piste — TODO 6 et 7

```
gx = (
    np.mean(champ[x + 1:x + r + 1,
                  y - r:y + r + 1])
    - np.mean(champ[x - r:x,
                    y - r:y + r + 1])
)
gy = (
    np.mean(champ[x - r:x + r + 1,
                  y + 1:y + r + 1])
    - np.mean(champ[x - r:x + r + 1,
                    y - r:y])
)
vecteur = np.array([gx, gy], dtype=float)
norme = np.linalg.norm(vecteur)
if norme == 0:
    return np.zeros(2)
return vecteur / norme
```

Retourner au nid — TODO 8

À vous

- Orienter une fourmi chargee vers le nid.

Retourner au nid — TODO 8

```
cible = direction_vers(NID, position)
bruit = np.random.normal(0, BRUIT_DIRECTION, 2)
nouvelle_direction = cible + 0.25 * bruit
```

Une fourmi chargée connaît le nid, mais son trajet reste légèrement irrégulier.

Explorer et suivre — TODO 9 et 10

À vous

- Combiner inertie, pheromones et exploration.
- Ajouter la repulsion du nid.

Explorer et suivre — TODO 9 et 10

```
nouvelle_direction = (directions[i]
    + FORCE_PHEROMONE * odeur + bruit)

nouvelle_direction += (FORCE_ELOIGNEMENT_NID
    * intensite * eloignement)
```

L'inertie, le hasard et l'odeur sont combinés. Près du nid, une répulsion évite que les fourmis restent regroupées au même endroit.

Avancer et rebondir horizontalement — TODO 11 et 12

À vous

- Memoriser la nouvelle direction unitaire.
- Completer les conditions des quatre rebonds.

Avancer et rebondir horizontalement — TODO 11 et 12

```
directions[i] = nouvelle_direction
positions[i] += VITESSE * directions[i]

if positions[i, 0] < 1:        # gauche
    positions[i, 0] = 1
    directions[i, 0] = -directions[i, 0]
if positions[i, 0] > TAILLE - 2:  # droite
    positions[i, 0] = TAILLE - 2
    directions[i, 0] = -directions[i, 0]
```

L'indice 0 désigne la coordonnée horizontale x.

Rebondir verticalement — TODO 12

À vous

- Completer les conditions des quatre rebonds.

Rebondir verticalement — TODO 12

```
if positions[i, 1] < 1:        # bas
    positions[i, 1] = 1
    directions[i, 1] = -directions[i, 1]
if positions[i, 1] > TAILLE - 2:  # haut
    positions[i, 1] = TAILLE - 2
    directions[i, 1] = -directions[i, 1]
```

L'indice 1 désigne y. Dans chaque cas, la composante correspondante de la direction change de signe.

Changer d'état — TODO 13 et 14

À vous

- Prendre la nourriture et initialiser le depot.
- Livrer, compter puis repartir vers l'exterieur.

Changer d'état — TODO 13 et 14

```
# A la nourriture
transporte[i] = True
reserve_pheromone[i] = DEPOT_PHEROMONE
directions[i] = direction_vers(NID, positions[i])

# Au nid : livrer puis faire demi-tour
transporte[i] = False
fourmis["livraisons"] += 1
directions[i] *= -1
```

Une petite mémoire directionnelle

Au retour, la fourmi apprend seulement le **cap opposé** à son arrivée. Elle le conserve temporairement en quittant le nid ; le gradient de phéromones corrige doucement sa direction.

Ce qu'elle ne connaît pas

Le programme ne lui donne jamais les coordonnées de la nourriture pendant la recherche.

Déposer une piste — TODO 15

À vous

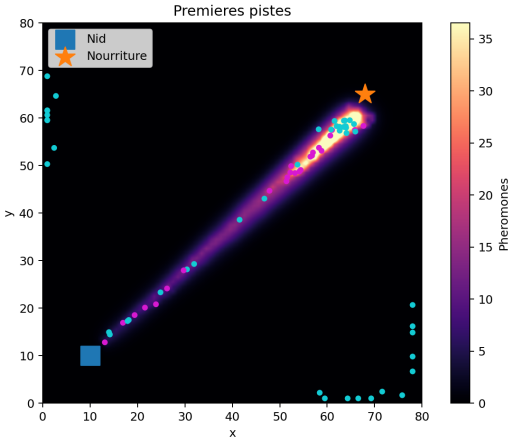
- Deposer la reserve puis la faire decroitre.

Déposer une piste — TODO 15

```
champ[x, y] = champ[x, y] + reserve_pheromone[i]
reserve_pheromone[i] *= DECROISSANCE_DEPOT
```

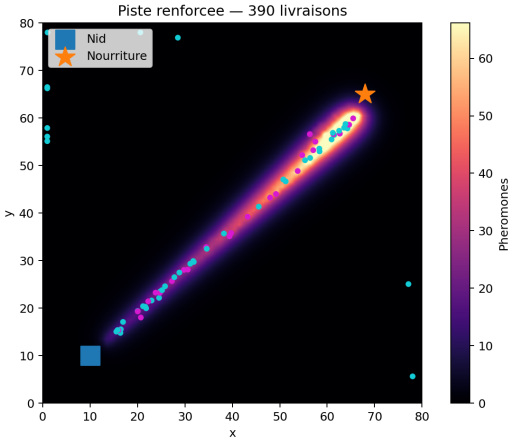
La réserve est forte près de la nourriture puis diminue vers le nid : le gradient pointe ainsi vers la source.

Premières pistes



Les premières découvertes sont aléatoires. Les retours chargés commencent à dessiner des traces que les autres fourmis peuvent rejoindre.

Une piste collective renforcée



Les passages fréquents renforcent la piste efficace. Les détours peu utilisés s'évaporent progressivement.

Expérimenter

Modifier un seul paramètre puis observer le nombre de livraisons :

- `EVAPORATION` : durée de vie des pistes ;
- `DIFFUSION` : largeur des pistes ;
- `FORCE_PHEROMONE` : attraction odorante ;
- `BRUIT_DIRECTION` : exploration aléatoire ;
- `NB_FOURMIS` : taille de la colonie.

Lancer le TP dans Spyder

1. Ouvrir `04-fourmis_pheromones.py`.
2. Choisir le backend **Automatic** ou **Qt**.
3. Redémarrer la console IPython après un changement de backend.
4. Compléter les TODO dans l'ordre et exécuter après chaque groupe.

Animation dynamique

`plt.show(block=True)` garde la fenêtre ouverte.

Concepts appris

- vecteurs et norme ;
- normalisation ;
- tableaux NumPy ;
- gradient discret ;
- diffusion et évaporation ;
- états booléens ;
- combinaison de forces ;
- émergence collective.

À vous de coder !